Hardware-aware boost + CAD-textured 3-D v0.95.0

Tuned to the machine: 16-core Alder Lake + RTX 4050. Use the cores for the CAD batch; graft the CAD look onto the WebGL model.

1 · PARALLEL CAD BATCH (the CPU-bound straggler, fixed)

make_cad.py was single-threaded → now multiprocessing.Pool

· auto-sized to cpu_count-1 (cap 12); --workers / --serial.
· renders are independent → distinct cache files, no contention.
· parent pre-filters cached parts (resumable).

MEASURED (16 cores): 120 parts 15.4s serial -> 5.26s on 12 workers = 2.9x

Full re-render (98k images, 3 tiers)

serial ~210 min

parallel ~72 min

(+ per-worker startup amortizes further at full scale)

GPU note: RTX 4050 already drives WebGL 3-D + GPU-tier OCR.

2 · CAD COLOUR + TEXTURE GRAFTED ONTO THE 3-D MODEL

cad_render.material_for()

colour + metal + material CLASS (metal/rubber/wood/plastic/ paint/brass) + a gl vector. route /api/cadmaterial.

→

gl3d.js shader

NEW klass uniform -> procedural texture: brushed metal, rubber speckle, wood rings, CARC orange-peel, brass. load/setKlass.

→

threed.html

applyCadMaterial(m) on open -> colour+class on the Interactive 3-D model. Rotate CAD stays flat steel (klass 0).

→

RESULT

the WebGL 3-D model now MATCHES the CAD image's colour + surface. graceful: a bad shader just falls back to SVG (no crash).

VERIFIED (host-side)

- VERIFY-BOOST.bat -> docs/boost_verify.log: cad_render / make_cad / viewer_app parse; gl3d.js passes node --check; both UI pages parse.
- Benchmark printed the 2.9x speedup above (bench_cad_parallel.py).
- CAD texture is a gl3d shader add with a graceful fallback (compile-fail -> SVG, never a crash) — and it compiles.
- Additive & rollbackable (R1): revert the pool in make_cad, the shader klass block, material_for + /api/cadmaterial, applyCadMaterial.